



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Kirana Cinta Mentari

NIM : 2428240078

Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Kirana Cinta Mentari
NPM	2428240078
Kelas	SI5B
Pertemuan Ke-	5
Topik / Judul Praktikum	RESTful API dengan Express.js: Routing, Middleware, dan CRUD Data Mahasiswa
Tanggal Pelaksanaan	24 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Memahami konsep RESTful API serta penggunaan metode HTTP (GET, POST, PUT) dan status code (200, 201, 400, 404) pada layanan web.
2. Mampu membangun server web sederhana menggunakan Express.js, meliputi routing dasar serta penggunaan middleware (logger custom, CORS, dan `express.json()`).
3. Mampu membangun server web sederhana menggunakan Express.js, meliputi routing dasar serta penggunaan middleware (logger custom, CORS, dan `express.json()`).
4. Mampu mengelola konfigurasi aplikasi menggunakan variabel lingkungan (`.env`) dengan library `dotenv` dan menguji endpoint API yang telah dibuat.

II. Dasar Teori

REST (*Representational State Transfer*) adalah gaya arsitektur perangkat lunak untuk sistem terdistribusi yang diperkenalkan oleh Fielding (2000). Pada RESTful API, setiap data atau sumber daya (*resource*), misalnya mahasiswa, diidentifikasi melalui URL dan dimanipulasi menggunakan metode HTTP standar, dengan pertukaran data yang umumnya berformat JSON. Layanan ini bersifat *stateless*, artinya setiap permintaan membawa seluruh informasi yang dibutuhkan server untuk memprosesnya. Metode HTTP yang lazim digunakan adalah GET untuk membaca data, POST untuk membuat data baru, PUT untuk memperbarui data, dan DELETE untuk menghapus data (Fielding et al., 2022). Server menyampaikan hasil pemrosesan melalui status code, misalnya 200 (berhasil), 201 (data dibuat) untuk data yang berhasil dibuat, 400 (permintaan tidak valid) untuk

permintaan yang tidak valid, dan 404 (data tidak ditemukan) untuk data yang tidak ditemukan.

Express.js adalah framework web yang minimalis untuk Node.js dan menyediakan mekanisme routing serta middleware (Express, n.d.). Routing menentukan cara aplikasi merespons kombinasi metode HTTP dan path tertentu. Data dari klien dapat diterima melalui route parameter (/mahasiswa/:id, dibaca dengan req.params), query string (?jurusan=..., dibaca dengan req.query), maupun request body (dibaca dengan req.body).

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript (versi sesuai hasil perintah node -v)
2	Visual Studio Code	Editor code untuk mengetik program
3	npm	Package manager bawaan Node.js
4	Cors, dotenv, nodemon	Versi 2.8.6 (middleware CORS), Versi 18.0.4 (pemuat variabel lingkungan), Versi 3.1.14 (devDependency, auto-restart server)
5	Postman	Pengujian endpoint API
6	Express.js	Versi 5.2.1 (framework web)
7	Git dan Github	Version control dan penyimpanan repository

IV. Langkah Kerja

1. Membuat folder project restful-api-ap, lalu menjalankan npm init untuk membuat package.json.
2. Memasang dependency yang dibutuhkan dengan perintah npm install express cors dotenv, serta npm install --save-dev nodemon untuk auto restart saat pengembangan. Pada package.json didefinisikan script start (node app.js) dan dev (nodemon app.js).
3. Membuat berkas .env berdasarkan .env.example yang berisi variabel PORT, API_KEY, CORS_ORIGIN, DB_URL, serta berkas .gitignore agar folder node_modules atau berkas .env tidak ikut diunggah ke repository.
4. Membuat berkas app.js yang memuat dotenv dengan require('dotenv').config(), mengimpor Express dan cors, lalu menentukan PORT dari variabel default 3000.
5. Membuat middleware custom logger yang mencatat waktu, metode, dan URL setiap request ke konsol, kemudian memanggil next(). Middleware didaftarkan dengan app.use() sebelum route, dilanjutkan dengan cors() (dari CORS_ORIGIN, metode GET, POST, PUT, DELETE) dan express.json() agar req.body dapat dibaca.
6. Membuat route dasar GET/ dan GET /profile, lalu menjalankan server dengan app.listen(PORT).
7. Menyiapkan data sementara berupa array mahasiswa dan variabel nextId sebagai penghitung id data baru. Data disimpan di memori sehingga hilang saat server di restart.
8. Membuat route GET /mahasiswa yang menampilkan seluruh data dan mendukung filter ?jurusan= melalui req.query, serta GET /mahasiswa/:id yang mencari satu data dengan find() dan mengembalikan status 404 bila tidak ditemukan.
9. Membuat route POST /mahasiswa/:id yang memvalidasi nama dan jurusan (status 4000 bila kosong), menambahkan data baru ke array dan mengembalikan status 201, serta route PUT /mahasiswa:id yang mencari data dengan findById(), menggabungkan data lama dengan req.body, dan mengembalikan status 404 bila id tidak ada.
10. Menjalankan server dengan npm run dev, menguji seluruh endpoint menggunakan postman atau browser, lalu mendokumentasikan hasilnya dengan tangkapan layar.
11. Mengunggah project ke repository GitHub menggunakan Git (git init, git add, git commit, git push).

V. Hasil dan Pembahasan

```
PS C:\Users\cinta\SI5B-Express\restful-api-app> npm run dev
npm notice run restful-api-app@1.0.0 dev
npm notice run nodemon app.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
◇ injected env (4) from .env
Server berjalan di http://localhost:4000
```

Gambar 1. Server Express berhasil berjalan di port 4000 menggunakan nodemon (`npm run dev`) dengan konfigurasi dari file `.env`

```
PS C:\Users\cinta\SI5B-Express\restful-api-app> npm run dev
Server berjalan di http://localhost:4000
[2026-09-30T16:03:07.205Z] GET /
[2026-09-30T16:03:17.955Z] GET /mahasiswa
[2026-09-30T16:03:27.503Z] GET /mahasiswa
[2026-09-30T16:03:53.033Z] PUT /mahasiswa
[2026-09-30T16:04:29.117Z] PUT /mahasiswa
[2026-09-30T16:04:33.908Z] PUT /mahasiswa/2
[2026-09-30T16:04:39.207Z] PUT /mahasiswa/1
[2026-09-30T16:04:44.224Z] PUT /mahasiswa
```

Gambar 2. Log middleware logger di terminal yang mencatat setiap request, yaitu PUT dan GET pada endpoint `/mahasiswa/:id`.

Pembahasan:

Gambar 1. Pada terminal terlihat bahwa aplikasi dijalankan menggunakan nodemon, sehingga server akan otomatis melakukan restart ketika terdapat perubahan pada kode. Server berhasil berjalan pada di <http://localhost:4000/>, port yang dibaca dari berkas `.env` (`process.env.PORT`), yaitu 4000 (dengan nilai default 3000 apabila variabel tidak tersedia).

Gambar 2. middleware logger berhasil mencatat setiap request yang diterima oleh server. Request yang tercatat terdiri dari metode `GET` dan `PUT` pada endpoint `/mahasiswa` serta `/mahasiswa/:id`. Pada log terlihat beberapa proses seperti `GET`

/mahasiswa, PUT /mahasiswa, PUT /mahasiswa/1, dan PUT /mahasiswa/2. Ini menunjukkan bahwa endpoint mahasiswa berhasil menerima dan memproses permintaan dari client, sedangkan middleware logger berfungsi untuk menampilkan informasi request ke terminal sehingga aktivitas API dapat dipantau selama aplikasi berjalan.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Berkas .env tidak ikut tersedia setelah proyek diunduh dari repository karena masuk dalam .gitignore, sehingga server berjalan di port default 3000 dan CORS_ORIGIN bernilai undefined.	Membuat berkas .env sendiri dengan menyalin isi .env.example lalu menyesuaikan nilainya.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, RESTful API sederhana berhasil dibangun menggunakan Express.js. API menerapkan metode HTTP GET, POST, dan PUT untuk mengelola data mahasiswa serta mengembalikan status code yang sesuai (200, 201, 400, dan 404), sehingga tujuan pertama dan ketiga tercapai.

Penggunaan routing dan middleware (logger custom, CORS, dan express.json()) menunjukkan bahwa urutan pendaftaran middleware berpengaruh pada alur request, dan pemanggilan next() diperlukan agar request diteruskan. Data dapat diterima melalui route parameter, query string, dan request body, sehingga tujuan kedua tercapai.

Konfigurasi aplikasi berhasil dipisahkan dari kode menggunakan dotenv dan berkas .env. Data yang disimpan di memori bersifat sementara sehingga pengembangan selanjutnya memerlukan basis data agar data tersimpan permanen.

Daftar Pustaka

- Express. (n.d.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc9110>

Lampiran

Tautan Repository GitHub:

https://github.com/kiranacintamentari2428240078/si5b_latihan_restful_KiranaCintaMentari

Tautan Deployment (jika ada):

<https://vercel.com/kiranacintamentari2428240078s-projects/si5b-latihan-restful-kirana-cinta-mentari>
